

Perbandingan Arsitektur Conductor

Analisis Throughput & Kinerja: ClickHouse + CouchDB vs. Elasticsearch + PostgreSQL

1. Ringkasan Eksekutif

Conductor menerapkan arsitektur *interface-based* yang memisahkan lapisan penyimpanan operasional (ExecutionDAO, MetadataDAO) dan lapisan pencarian/analitik (IndexDAO). Evaluasi ini membandingkan arsitektur modern berbasis NoSQL/OLAP (ClickHouse + CouchDB) dengan pasangan standar industri yang umum digunakan (Elasticsearch + PostgreSQL).

2. Tabel Perbandingan Throughput & Latensi

Karakteristik / Metrik	ClickHouse + CouchDB	Elasticsearch + PostgreSQL	Analisis pada Conductor
Write / Ingestion Throughput (Log & Index)	Sangat Tinggi ~50.000 – 150.000+ ops/s	Sedang – Tinggi ~10.000 – 30.000 ops/s	ClickHouse menggunakan columnar <i>MergeTree</i> dengan batch append cepat tanpa beban tokenisasi invert-index.
Point Mutation / State Updates (ExecutionDAO)	Sedang – Tinggi ~2.000 – 8.000 ops/s	Tinggi – Sangat Tinggi ~8.000 – 25.000+ ops/s	PostgreSQL unggul dalam in-place row update dengan WAL & pooled JDBC. CouchDB menggunakan HTTP REST MVCC overhead.
Pencarian Bebas (Full-Text Search)	Sedang (Regex / hasToken / Bloom)	Sangat Tinggi (Native Inverted Index)	Elasticsearch memiliki Lucene tokenizer yang unggul untuk pencarian substring acak dalam payload arbitrary JSON.
Query Agregasi & Analitik Historis	Ekstrem Cepat (Milidetik / Ratusan Juta Baris)	Sedang (Intensif RAM & Garbage Coll.)	ClickHouse membaca data per-kolom terkompresi dengan SIMD vektor, unggul telak untuk analitik SLA & metrik task.
Latensi Indeks (Real-time Delay)	Rendah – Sedang (Tergantung batch buffer)	Sedang (Default refresh_interval 1s)	Kedua engine memanfaatkan antrean asinkron Conductor untuk membongkar beban dari alur eksekusi utama.
Efisiensi Kompresi & Ruang Disk	Sangat Tinggi (Rasio kompresi 5x – 10x)	Rendah – Sedang (1.5x – 3x data asli)	Kompresi LZ4/ZSTD kolom ClickHouse sangat hemat dibanding term dictionary & index Lucene di Elasticsearch.
Konsumsi Memori (RAM / Heap)	Rendah – Menengah (Engine C++ efisien)	Tinggi – Sangat Tinggi (JVM Heap 16GB – 32GB+)	Elasticsearch rentan terhadap garbage collection pause pada beban tinggi. ClickHouse mengelola memori secara ramping.

3. Analisis Kelebihan & Trade-Off

A. Konfigurasi ClickHouse + CouchDB (Modern NoSQL & OLAP)

- **Native JSON Workflow:** Semua model Conductor (Workflow, Task, Def) adalah JSON murni, disimpan secara natural di CouchDB tanpa perlu Flyway database migration.
- **Efisiensi Skala Masif:** ClickHouse mampu menelan puluhan ribu log eksekusi per detik dengan penggunaan disk dan RAM yang jauh lebih kecil.
- **Trade-Off:** Point-update task/workflow di CouchDB via HTTP REST memiliki latensi per-request sedikit lebih tinggi dibanding koneksi socket biner relasional PostgreSQL.

B. Konfigurasi Elasticsearch + PostgreSQL (Teruji & Ekosistem Matang)

- **Throughput Transaksional Cepat:** PostgreSQL sangat cepat dalam mutasi baris individual (OLTP) berkat WAL in-memory dan connection pooling.
- **Pencarian Fleksibel:** Elasticsearch mendukung pencarian teks bebas (full-text) mendalam di seluruh bagian payload input/output JSON.
- *Trade-Off:* Biaya operasional tinggi; cluster Elasticsearch membutuhkan RAM besar dan kapasitas disk berlebih untuk inverted index.

4. Panduan Pemilihan Rekomendasi

PILIH CLICKHOUSE + COUCHDB JIKA:

1. Menghasilkan jutaan task execution log per hari.
2. Ingin memangkas biaya infrastruktur (RAM & Disk).
3. Mengutamakan query analitik & SLA reporting cepat.
4. Memprioritaskan kemudahan schema-less JSON.

PILIH ELASTICSEARCH + POSTGRESQL JIKA:

1. Membutuhkan TPS eksekusi workflow sangat tinggi.
2. Sangat bergantung pada pencarian full-text arbitrary.
3. Sudah memiliki cluster PostgreSQL & ES yang stabil.
4. Memerlukan kepatuhan ACID relasional ketat.

5. Status Modul di Codebase Conductor

Kedua modul baru telah berhasil dikompilasi dan diintegrasikan ke dalam Conductor Quarkus:

- **conductor-couchdb-persistence:** Menyediakan CouchDbExecutionDAO, CouchDbMetadataDAO, CouchDbPollDataDAO, dan CouchDbEventHandlerDAO.
- **conductor-clickhouse-persistence:** Menyediakan ClickHouseIndexDAO (dengan *ReplacingMergeTree* & *MergeTree*) serta dukungan primary DB ClickHouseExecutionDAO & ClickHouseMetadataDAO.